DY @ N3LO, triple-real: running the pipeline with `run_all.py`

Code_v2/Scripts

October 15, 2026

Contents

1 What runs where 1

2 Setup (once per machine) 1

3 Before a production run 1

4 Production run 2

5 Monitoring 2

6 Problems 2

7 After a run: checks 3

8 Integrals to reduce 3

9 Other tools 3

10 Timings so far (for orientation) 3

1 What runs where

- One command runs the whole pipeline for a set of channels: QGRAF → 01 (parse, colour) → 02 (groups of 10 diagrams) → 03 (conjugates) → 04 (square) → 05 (projector, partial fractions, families, integrals) → 06 (sum over pairs, relabelling to the 156 independent families). One global job queue, most expensive jobs first; a job's step-05 jobs start as soon as its step-04 job is done.
- Sets: `gluons` = `physical_qqbggg` (7 channels), `quarks` = the other 42 channels, `all`; or `-channels A,B,...`.
- Plan: gluons on **newton4**, quarks on **newton1**. Each machine has its own clone in `/scratch/ge45tuy/DY_P2B` (local disk) and its own state in `Code_v2/Scripts/run_state/<set>/`. The home directory `/home/t30/...` is shared by both machines.
- Results stay in each channel folder `Scripts/<group>/<P>/<P>_LO/tmp/`: step 04 `squared_diagrams/`, step 05 `squared_contracted/`, step 06 `unreduced/<piece>/unreduced<PROJ>.ff` (and `.m`). None of these is in git.

2 Setup (once per machine)

```
cd /scratch/ge45tuy && git clone https://github.com/cesarecarlo-mella/DY_P2B.git    # or: cd DY_P2B &&
    git pull
cd /scratch/ge45tuy/DY_P2B/Code_v2
source setenv.sh                                  # sets FORMPATH; needed in every new shell / tmux
    window
echo $FORMPATH $QGRAF
```

In `~/.bashrc` (shared home, so once for both machines):

```
export QGRAF=/home/t30/all/ge45tuy/Software/qgraf-3.4.2/qgraf    # qgraf is only an alias in the shell
```

Optional environment variables: `FORM` (default `form`; e.g. `FORM=$HOME/opt/form5/bin/form`), `SKIPAA` (default 1: the pieces aa and a2a2 are not computed, aa = vv and a2a2 = v2v2 in the combination; `SKIPAA=0` computes them).

Updating the code: `git push` on the laptop, then on the machines

```
cd /scratch/ge45tuy/DY_P2B && git status --short | grep -v '^??' | head && git pull
```

Safe also during a run: results, `run_state/`, `formtmp` and `integrals/` are not tracked by git. Deleted `.gitkeep` files in `git status` are harmless.

3 Before a production run

```
python3 Scripts/run_all.py selftest --jobs 8      # FORM on this machine vs reference numbers: must end
    with SELFTEST PASSED
mkdir -p /scratch/ge45tuy/formtmp                 # FORM temporary files: on /scratch, NEVER /tmp (that
    is RAM)
```

Optional pilot (one small channel, about 20 minutes):

```
python3 Scripts/run_all.py run --channels Vqqbccbg --jobs 12 --form-tmp /scratch/ge45tuy/formtmp
```

4 Production run

In a tmux session (survives logout; detach with `Ctrl-b d`, come back with `tmux attach -t dy`):

```
tmux new -s dy
cd /scratch/ge45tuy/DY_P2B/Code_v2 && source setenv.sh
python3 Scripts/run_all.py run --set gluons --jobs 100 --form-tmp /scratch/ge45tuy/formtmp 2>&1 | tee
    gluons.log
#   newton1: the same with --set quarks ... | tee quarks.log
```

Useful options of `run`:

- `-jobs N`: parallel FORM jobs.
- `-min-free-gb 100`: no new job while the machine has less available memory.
- `-max-job-gb 60`: a FORM job above this memory is stopped and marked failed.
- `-nice 10`: priority of the FORM jobs.
- `-steps qgraf,01,02,03,04,05,06`: restrict the steps (default all).
- `-state DIR`: other state directory (default `Scripts/run_state/<set>`).

Restart / resume: run the same command again. Finished jobs are skipped; failed and interrupted ones run again. Redoing a preparation step (qgraf, 01–03) of a channel invalidates its later steps.

Do not delete `/scratch/ge45tuy/formtmp` while a run is going (running jobs keep their sort files there).

5 Monitoring

From any shell on the same machine (`status` needs no `setenv.sh`):

```
cd /scratch/ge45tuy/DY_P2B/Code_v2
watch -n 120 python3 Scripts/run_all.py status --set gluons          # overview, refreshed every 2 min
python3 Scripts/run_all.py status --set gluons --running             # + every running job: time,
    memory
tail -f Scripts/run_state/gluons/jobs.csv                            # one line per finished job
tail Scripts/run_state/gluons/system.csv                             # memory, load, disk every 30 s
tail -f gluons.log                                                   # what the run prints
htop ; free -g                                                       # the machine as a whole
```

Reading `status`:

- `prep`: last preparation step done (qgraf, 01, 02, 03).
- `04 done/all`: group pairs (groups × groups).
- `05 done/known`: step-05 jobs; "known" = non-zero pieces of the finished step-04 pairs (grows while step 04 runs; 3 pieces per pair in the gluon channels, up to 14 in the Q2 channels).
- `06 done`: summed pieces.
- `running`, `FAILED`; at the end the failure reasons.
- The time estimate is pessimistic early on (the most expensive jobs run first).

Live stage of the running step-05 jobs, and the size of their temporary files:

```
cd Scripts; for f in physical_qqbggg/*/*_LO/utils/job_*/form.log; do echo "$(basename $(dirname $f)): $(
    grep '~~~' $f | tail -1)"; done
du -sh /scratch/ge45tuy/formtmp/formtmp/* | sort -h | tail -5
```

6 Problems

- **Failed jobs**: reason at the end of `status`, details in `run_state/<set>/failed/<job>.json`; the FORM log is kept: step 04 `<P>_LO/utils/tmp_fld/tmp_<i_j>/form.log`, step 05 `<P>_LO/utils/job_<piece>_<PROJ>_<i_j>/for`, steps 01–03 and 06 `<P>_LO/run<step>.log` / `utils/sum_squared_diags_*.log`. After fixing the cause: run the same command again.
- **Stop a run**: `Ctrl-C` in its tmux window (it stops its FORM jobs). If FORM jobs are left over (`pgrep -u ge45tuy -a form`): `pkill -u ge45tuy -x form` — only on a machine that runs no other set.
- **Clean temporary files** (only with no run going on that machine):

```
rm -rf /scratch/ge45tuy/formtmp
cd /scratch/ge45tuy/DY_P2B/Code_v2/Scripts
rm -rf */*/*_LO/utils/tmp_fld */*/*_LO/utils/job_* */*/*_LO/utils/formtmp */*/*_LO/tmp_folder
rm -f  */*/*_LO/utils/sum_squared_diags_*.frm */*/*_LO/utils/sum_squared_diags_*.log
```

- **Start from scratch** for a set: delete `Scripts/run_state/<set>` (all jobs then run again).

7 After a run: checks

```
source setenv.sh
python3 Scripts/run_all.py check  --set gluons          # completeness, leftovers, outputs made with
    older files
python3 Scripts/run_all.py verify --set gluons --n 2    # numerical check of 2 random step-05 results
    per channel
```

`check` reads all result files: no missing pair or piece, nothing unprocessed in the step-05 results (no `Prop`, `D/DD`, `TOPO`, scalar products, `m2`, ...; cut indices of every `INT` = 1), no dependent family after step 06, no output made with an older template or family file. `verify` evaluates the step-04 input (contracted with the projector) and the step-05 result at a random rational point on the cut surface and compares them exactly; it prints `IDENTICAL` or `DIFFERENT`.

8 Integrals to reduce

```
python3 Scripts/run_all.py integrals --set gluons      # newton4 -> ~/integrals/gluons/
python3 Scripts/run_all.py integrals --set quarks      # newton1 -> ~/integrals/quarks/
python3 Scripts/run_all.py integrals --merge           # either machine: all collections in ~/integrals
    -> ~/integrals/all/
```

`~/integrals` is in the home directory shared by both machines (other place: `-out DIR`, or the base folder with `export DY_INTEGRALS=...`; `-merge DIR1 DIR2` merges just these folders). Output folder: `integrals.m` (`{INT[topo55, 1,1,1,1,1,-3,...], ...}`), `integrals.txt` and `kira/topoN.txt` (`topo55[1,1,1,1,1,-3,...]`, one per line), `summary.txt` (per family: integrals, sectors, max dots, max numerator power), `sources.txt`. `-step 05`: from the per-pair step-05 results (all 921 families, no relabelling).

Families for the reduction: `Code_v2/Families/reduction/` — `integralfamilies_independent.rrr.m` and `independentFam` (the 156 independent families, p4 = −r1, p5 = −r2, p6 = −r3, propagators in the order of the `INT` indices), `integralfamilies_all.rrr.m` (all 921), `relabelling_INT.m` / `symm_loop_crossings.inc` (rules to the independent families), `README.md`.

9 Other tools

- Diagrams: `python3 Scripts/diagrams/draw_diagrams.py [channel ...]` → `Scripts/diagrams/diagrams.pdf` (one representative diagram per channel; needs the QGRAF outputs).
- Families: `Code_v2/Families/` — `coverage.py` (every propagator set after partial fractions in a family), `reduction_export.py`, `check_relabelling.wl`; details in `Scripts/README.md`.
- Help: `python3 Scripts/run_all.py -help`, `python3 Scripts/run_all.py run -help`, ...

10 Timings so far (for orientation)

- Laptop, FORM 5.0: gluon 10×10 pair step 04 about 40 min; step 05 7–15 min per piece.
- newton4, FORM 4.2.1, 100 jobs: gluon set (175 pairs, 525 step-05 jobs) done overnight; step 04 mean 46 min, step 05 up to several hours per piece for the largest pairs (6–12 GB of FORM sort files per job).
- Pending: test FORM 5.0 and larger FORM buffers on newton for step 05.